```python
import tkinter as tk
from tkinter import ttk, messagebox, filedialog
from collections import deque
from datetime import datetime
import re


def check_palindrome(text):
    """
    Checks whether a string is a palindrome using
    Stack (list) and Queue (deque).
    Spaces, punctuation and letter case are ignored.
    """

    cleaned = re.sub(r"[^a-zA-Z0-9]", "", text).lower()

    stack = []
    queue = deque()

    # Step 1: Insert characters into stack and queue
    for char in cleaned:
        stack.append(char)
        queue.append(char)

    # Step 2: Compare stack and queue characters
    steps = []
    comparisons = 0
    is_palindrome = True

    while stack and queue:
        stack_char = stack.pop()
        queue_char = queue.popleft()
        comparisons += 1

        steps.append(
            f"Step {comparisons}: "
            f"Stack pop = '{stack_char}', "
            f"Queue remove = '{queue_char}'"
        )

        if stack_char != queue_char:
            is_palindrome = False
            steps.append("Mismatch found. Not a palindrome.")
            break

    if is_palindrome:
```

```
        steps.append("All compared characters match.")

    return cleaned, is_palindrome, steps, comparisons


class PalindromeApp:
    def __init__(self, root):
        self.root = root
        self.root.title("DSL Mini Project - Palindrome Checker")
        self.root.geometry("850x700")
        self.root.minsize(700, 600)
        self.root.configure(bg="#eef2f7")

        self.last_report = ""

        self.create_gui()

    def create_gui(self):
        # Header
        header = tk.Frame(self.root, bg="#172554", pady=18)
        header.pack(fill="x")

        tk.Label(
            header,
            text="PALINDROME CHECKER",
            font=("Arial", 23, "bold"),
            fg="white",
            bg="#172554"
        ).pack()

        tk.Label(
            header,
            text="Data Structures Laboratory | Stack and Queue",
            font=("Arial", 11),
            fg="#bfdbfe",
            bg="#172554"
        ).pack(pady=(5, 0))

        # Main container
        main = tk.Frame(self.root, bg="#eef2f7", padx=25, pady=18)
        main.pack(fill="both", expand=True)

        tk.Label(
            main,
            text="Enter a word or sentence:",
            font=("Arial", 12, "bold"),
```

```
    bg="#eef2f7"
).pack(anchor="w")

# Input field
input_frame = tk.Frame(main, bg="#eef2f7")
input_frame.pack(fill="x", pady=8)

self.entry = ttk.Entry(
    input_frame,
    font=("Arial", 14)
)
self.entry.pack(side="left", fill="x", expand=True, ipady=7)
self.entry.bind("<Return>", lambda event: self.run_check())

ttk.Button(
    input_frame,
    text="Check",
    command=self.run_check
).pack(side="left", padx=(10, 0), ipady=5)

# Example buttons
examples = tk.Frame(main, bg="#eef2f7")
examples.pack(anchor="w", pady=(0, 10))

tk.Label(
    examples,
    text="Try an example:",
    bg="#eef2f7",
    font=("Arial", 10)
).pack(side="left", padx=(0, 8))

for word in ["MADAM", "LEVEL", "HELLO", "NURSESRUN"]:
    ttk.Button(
        examples,
        text=word,
        command=lambda w=word: self.load_example(w)
    ).pack(side="left", padx=3)

# Result panel
self.result_label = tk.Label(
    main,
    text="Enter text and click Check",
    font=("Arial", 15, "bold"),
    bg="white",
    fg="#334155",
    pady=15,
```

```
        relief="groove"
    )
    self.result_label.pack(fill="x", pady=8)

    self.cleaned_label = tk.Label(
        main,
        text="Normalized input: -",
        font=("Arial", 10),
        bg="#eef2f7",
        anchor="w"
    )
    self.cleaned_label.pack(fill="x", pady=3)

    # Data structure visualization
    visual_frame = tk.Frame(main, bg="#eef2f7")
    visual_frame.pack(fill="x", pady=10)

    left = tk.LabelFrame(
        visual_frame,
        text="STACK (LIFO)",
        font=("Arial", 11, "bold"),
        bg="white",
        padx=10,
        pady=8
    )
    left.pack(side="left", fill="both", expand=True, padx=(0, 7))

    right = tk.LabelFrame(
        visual_frame,
        text="QUEUE (FIFO)",
        font=("Arial", 11, "bold"),
        bg="white",
        padx=10,
        pady=8
    )
    right.pack(side="left", fill="both", expand=True, padx=(7, 0))

    self.stack_label = tk.Label(
        left,
        text="Empty",
        bg="white",
        fg="#1d4ed8",
        font=("Courier New", 12),
        wraplength=320,
        justify="left"
    )
```

```
    self.stack_label.pack(anchor="w", fill="x")

    self.queue_label = tk.Label(
        right,
        text="Empty",
        bg="white",
        fg="#047857",
        font=("Courier New", 12),
        wraplength=320,
        justify="left"
    )
    self.queue_label.pack(anchor="w", fill="x")

    # Steps output
    tk.Label(
        main,
        text="Algorithm Execution / Step Count",
        font=("Arial", 12, "bold"),
        bg="#eef2f7"
    ).pack(anchor="w", pady=(8, 4))

    output_frame = tk.Frame(main)
    output_frame.pack(fill="both", expand=True)

    scrollbar = ttk.Scrollbar(output_frame)
    scrollbar.pack(side="right", fill="y")

    self.output = tk.Text(
        output_frame,
        height=7,
        font=("Courier New", 10),
        wrap="word",
        yscrollcommand=scrollbar.set,
        state="disabled"
    )
    self.output.pack(side="left", fill="both", expand=True)
    scrollbar.config(command=self.output.yview)

    # Buttons
    button_frame = tk.Frame(main, bg="#eef2f7")
    button_frame.pack(fill="x", pady=12)

    ttk.Button(
        button_frame,
        text="Clear",
        command=self.clear_all
```

```
    ).pack(side="left", padx=(0, 8))

    ttk.Button(
        button_frame,
        text="Save Project Report",
        command=self.save_report
    ).pack(side="left", padx=8)

    ttk.Button(
        button_frame,
        text="Exit",
        command=self.root.destroy
    ).pack(side="right")

    tk.Label(
        main,
        text="Developed using Python and Tkinter",
        font=("Arial", 9, "italic"),
        bg="#eef2f7",
        fg="#64748b"
    ).pack(pady=(2, 0))

def load_example(self, word):
    self.entry.delete(0, tk.END)
    self.entry.insert(0, word)
    self.run_check()

def run_check(self):
    text = self.entry.get()

    if not text.strip():
        messagebox.showwarning(
            "Input Required",
            "Please enter a word or sentence."
        )
        return

    cleaned, result, steps, comparisons = check_palindrome(text)


    if not cleaned:
        messagebox.showwarning(
            "Invalid Input",
            "Enter at least one letter or digit."
        )
        return
```

```
if result:
    self.result_label.config(
        text="PALINDROME",
        fg="#15803d",
        bg="#dcfce7"
    )
else:
    self.result_label.config(
        text="NOT A PALINDROME",
        fg="#b91c1c",
        bg="#fee2e2"
    )

self.cleaned_label.config(
    text=f"Normalized input: {cleaned}"
)


stack_view = "TOP -> " + " | ".join(reversed(cleaned))
queue_view = "FRONT -> " + " | ".join(cleaned) + " <- REAR"

self.stack_label.config(text=stack_view)
self.queue_label.config(text=queue_view)

# Complexity and step count
n = len(cleaned)
time_complexity = "O(n)"
space_complexity = "O(n)"

report_lines = [
    "PALINDROME CHECKER - EXECUTION REPORT",
    "=" * 43,
    f"Original input: {text}",
    f"Normalized input: {cleaned}",
    f"Result: {'Palindrome' if result else 'Not a palindrome'}",
    "",
    "ALGORITHM STEPS:",
    "1. Read input from the user.",
    "2. Remove spaces and punctuation; convert to lowercase.",
    "3. Push every character onto the stack.",
    "4. Enqueue every character into the queue.",
    "5. Pop from stack and dequeue from queue.",
    "6. Compare the removed characters.",
    "7. If a mismatch occurs, report not a palindrome.",
    "8. Otherwise, report palindrome.",
```

```
            "",
            "COMPARISON LOG:"
        ]

        report_lines.extend(steps)
        report_lines.extend([
            "",
            "ANALYSIS:",
            f"Input length after normalization (n): {n}",
            f"Character comparisons performed: {comparisons}",
            f"Time complexity: {time_complexity}",
            f"Auxiliary space complexity: {space_complexity}",
            "Maximum character comparisons: ceil(n / 2)",
            "Stack push operations: n",
            "Queue enqueue operations: n",
            f"Stack pop operations: {comparisons}",
            f"Queue dequeue operations: {comparisons}",
        ])

        self.last_report = "\n".join(report_lines)

        self.output.config(state="normal")
        self.output.delete("1.0", tk.END)
        self.output.insert("1.0", self.last_report)
        self.output.config(state="disabled")

    def clear_all(self):
        self.entry.delete(0, tk.END)

        self.result_label.config(
            text="Enter text and click Check",
            fg="#334155",
            bg="white"
        )
        self.cleaned_label.config(text="Normalized input: -")
        self.stack_label.config(text="Empty")
        self.queue_label.config(text="Empty")

        self.output.config(state="normal")
        self.output.delete("1.0", tk.END)
        self.output.config(state="disabled")

        self.last_report = ""

    def save_report(self):
        if not self.last_report:
```

```
            messagebox.showinfo(
                "No Report",
                "Check a word first to generate an execution report."
            )
            return

        filename = filedialog.asksaveasfilename(
            defaultextension=".txt",
            filetypes=[
                ("Text files", "*.txt"),
                ("All files", "*.*")
            ],
            initialfile="Palindrome_Project_Report.txt"
        )

        if filename:
            with open(filename, "w", encoding="utf-8") as file:
                file.write("DSL MINI PROJECT\n")
                file.write("Palindrome Checker using Stack and Queue\n")
                file.write(f"Generated on: {datetime.now()}\n\n")
                file.write(self.last_report)

            messagebox.showinfo(
                "Saved",
                "Execution report saved successfully."
            )


if __name__ == "__main__":
    root = tk.Tk()
    app = PalindromeApp(root)
    root.mainloop()
```